

SGR explicado — cómo funciona el sistema que construimos

Documento generado desde `estado-del-sistema.md` del repositorio

Sistema de Gestión de Resultados · Delegaciones municipales de La Serena Equipo Origami SpA · Proyecto Integrador INACAP · 11 de septiembre de 2026

Este documento es una **visita guiada**, no una especificación. Sirve para entender qué hace el sistema, cómo está armado y dónde está cada cosa, sin tener que leer los siete documentos de la entrega.

Todo lo que se nombra acá está en el repositorio y **cada enlace de este PDF abre el archivo en GitHub**.

👉 **Repositorio:** <<https://github.com/TIbacache/matriz-sgr>>

1. Lo primero: míralo funcionando, sin instalar nada

No hace falta levantar nada para ver el sistema. Las 17 pantallas están congeladas en archivos que se abren con doble clic.

1. Descarga `mockup-17-pantallas.zip` (0,87 MB).
2. Descomprímelo.
3. Abre cualquier `.html` con doble clic.

Se ve **exactamente** como la aplicación real, sin servidor, sin base de datos y sin internet: el CSS, las fuentes, las imágenes y los gráficos viajan dentro del archivo. Los botones no responden —es una fotografía navegable—, y cada uno lo dice en una franja al pie.

Si prefieres verlas como imagen sin descargar nada, están una por una en [docs/mockups/](#), con su explicación.

Empieza por `03-ficha.html`. Es la pantalla más importante del sistema: ahí se ve casi todo lo que hace.

2. Qué problema resuelve

Las delegaciones municipales de La Serena registran su trabajo en una planilla compartida de Google Sheets. El problema no es la planilla: es lo que **no puede hacer**.

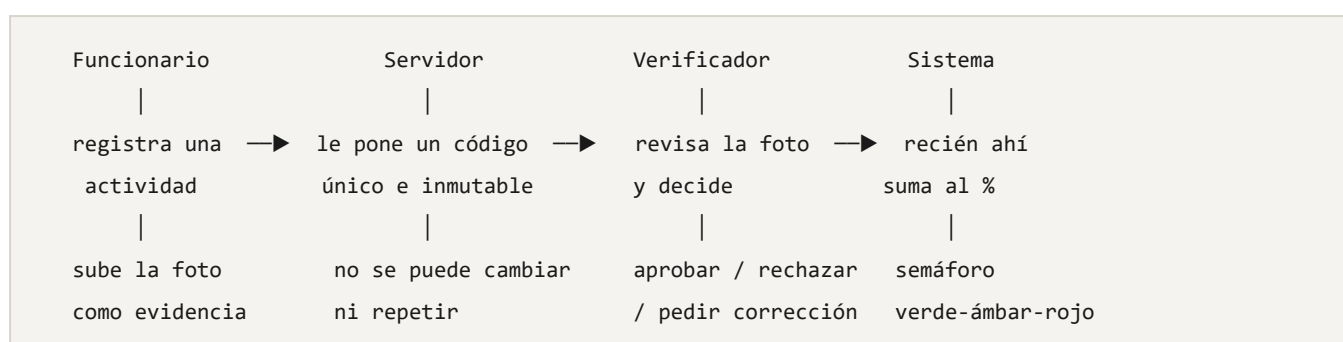
El cliente lo resumió con un caso concreto: **un niño pidió el mismo regalo de Navidad en cinco delegaciones distintas y nadie lo detectó**. Cada delegación llevaba su propio libro, la persona atendida no existía como dato del sistema, y cruzar la información significaba abrir cinco archivos a mano.

De ahí salen las tres cosas que el sistema tiene que lograr:

	Qué	Cómo lo resuelve SGR
1	Medir el desempeño de forma comparable	Metas y ponderadores por funcionario, ítem y período, con un solo motor de cálculo y un semáforo con umbrales configurables
2	Que la medición tenga respaldo	Una actividad solo suma cuando otra persona valida su evidencia , con código único e inmutable
3	Seguir a la persona atendida entre delegaciones	El RUT es único por organización, no por delegación , así que la duplicidad es una propiedad del modelo y no un informe posterior

3. La cadena que hay que entender

Si te quedas con una sola idea del sistema, que sea esta:



Nada suma solo porque alguien lo escribió. Esa es la regla que sostiene todo el sistema (se llama RN-009 en la especificación), y tiene dos consecuencias que conviene saber defender:

- **Nadie valida lo suyo.** Aunque tu rol te permita validar, el servidor rechaza tu propia evidencia. Si no fuera así, el «1» que suma al puntaje lo pondría la misma persona que lo pide.
- **Una aprobación no se deshace.** Ya sumó. Para corregir, se **anula la actividad con un motivo escrito**, que queda en la bitácora.

Cómo se calcula el porcentaje

1. Cada **cargo** tiene sus **ítems medidos** (al Apoyo Administrativo se le miden «Atención de usuario», «Llamadas preventivas», «Informe de inventarios»...).
2. A cada funcionario se le fija una **meta** y un **ponderador** por ítem. **Los ponderadores de una persona tienen que sumar 100% exacto.**
3. El avance de un ítem son sus **actividades validadas**, dividido por la meta.
4. El cumplimiento de la persona es la suma ponderada, con un **tope configurable** (hoy 150%) para que un ítem disparado no tape el resto.
5. El **objetivo al día** se calcula según cuántos días del período han pasado: a mitad de período se espera la mitad de la meta.
6. La **delegación** es el **promedio simple** de su gente.

⚠ Una delegación **sin nadie con metas configuradas no cumple 0%: no tiene medición**. Un 0% diría que trabajaron y no cumplieron; la verdad es que no hay nada que medir. En los datos de prueba, **La Pampa** está así a propósito para poder mostrar ese estado.

4. Quién es quién: los seis roles

El sistema tiene seis perfiles, que son los seis actores que pide el PDF del proyecto. **El nombre técnico no coincide con el municipal**, y esa equivalencia hay que tenerla a mano:

Se le dice	En el código	Qué puede hacer
Administrador	admin	Todo: delegaciones, usuarios, cargos, catálogos, períodos, metas
Coordinador	supervisor	Supervisa transversalmente, configura metas, ve el panel de actividad
Delegado (jefatura)	gerente	Su delegación: consulta, asigna y revisa compromisos
Funcionario	usuario	Registra actividades, compromisos, atenciones y evidencias
Verificador	verificador	Solo valida evidencias. No ve el tubo ni las fichas de vecinos
Usuario de consulta	consulta	Tableros e informes. No modifica nada

Los roles no son una jerarquía, y eso es una decisión de diseño que está escrita: son conjuntos que **se solapan**. El Verificador valida pero no ve el tubo; el Coordinador y el Administrador comparten el panel de actividad sin que uno herede del otro. Por eso el diagrama de clases **no dibuja herencia**: afirmarla sería decir algo que el código no cumple.

Dos reglas de visibilidad que no son de interfaz, son legales

El sistema trata datos personales de vecinos y de desempeño de funcionarios de un organismo público. Aplican la **Ley 21.663** de ciberseguridad y las **Leyes 19.628 / 21.719** de datos personales: finalidad, proporcionalidad, mínimo privilegio y trazabilidad.

- **El libro de cada delegación es privado de su equipo**. El semáforo consolidado lo ve todo el mundo; el detalle, no.
- **La ficha del vecino tiene alcance reducido**. Para un funcionario, una atención de otra delegación aparece con su fecha, su delegación y su tipo — el detalle queda reservado. Y **abrir una ficha queda registrado**, porque acceder a un dato personal también se audita, no solo modificarlo.

5. Las ocho pantallas, una por una

#	Pantalla	Ruta	Para qué sirve
1	<u>Ingreso</u>	/login	El Faro Monumental de La Serena, que gira e ilumina el mar
2	<u>Tubo de trabajo</u>	/	La agenda colectiva de la delegación: un kanban con arrastre, tiempo real y presencia

#	Pantalla	Ruta	Para qué sirve
3	<u>Ficha personal</u>	/ficha	La más importante. Cada funcionario ve su medición y registra su trabajo
4	<u>Bandeja del verificador</u>	/verificacion	La cola de evidencias por revisar: foto grande, tres decisiones, atajos de teclado
5	<u>Configuración de metas</u>	/metas	Qué se le mide a cada persona y con qué peso
6	<u>Ficha del vecino</u>	/vecinos	El control que el cliente vino a buscar: el historial cruzando delegaciones
7	<u>Tablero de control</u>	/dashboard	Semáforo por delegación, avance por área, proyección al cierre
8	<u>Control de actividad</u>	/actividad	Quién registró, quién no , y quién está conectado ahora

Lo que hay que saber de cada una

Tubo de trabajo. Es kanban de verdad: se arrastra una tarjeta y se mueve para todos los que tengan la pantalla abierta, en el momento. Si dos personas mueven la misma tarjeta a la vez, **el segundo recibe un aviso en pantalla** en vez de que su cambio se pierda en silencio.

Ficha personal. Arriba el semáforo con el cumplimiento del período y el objetivo al día; al medio la tabla de ítems con meta, avance y ponderador; abajo el registro del día a día, donde se escribe la actividad y se sube la foto. Una actividad aparece con su estado: *validada*, *en revisión*, *anulada*.

Bandeja del verificador. Las tres decisiones —aprobar, pedir corrección, rechazar— están **equidistantes y del mismo tamaño**: rechazar es tan legítimo como aprobar y debe costar lo mismo. Las dos últimas exigen una observación escrita: rechazar sin decir por qué no es trazabilidad.

Configuración de metas. El totalizador del 100% está **siempre visible**, y dice en todo momento «cuadrado en 100%», «falta 15%» o «se pasa por 8%». Lo que ya acumuló puntaje no se puede quitar.

Ficha del vecino. Se busca por RUT o por nombre y se ve el historial **cruzando delegaciones**, con un aviso ámbar cuando hay atenciones del mismo tipo en distintas delegaciones dentro de una ventana configurable. El aviso **informa: no bloquea ni acusa** — puede ser perfectamente correcto que dos delegaciones atiendan a la misma persona.

Tablero de control. Los gráficos leen los colores vivos de la aplicación, así que siguen el tema claro/oscuro solos.

Control de actividad. La tabla se ordena por **quien necesita atención primero**, no alfabéticamente, y separa lo *registrado* de lo *validado*: quien subió cuarenta actividades que esperan al verificador **sí está registrando**. La finalidad está escrita en la propia pantalla: **acompañar a quien se está quedando atrás, no vigilar**.

6. Qué pasa cuando algo sale mal

Nueve pantallas más muestran los caminos que no son el feliz. **Ninguna está simulada**: el generador abre la aplicación, la opera y captura lo que el servidor respondió de verdad.

Situación	Verla aquí
Un dato mal escrito	09-alt-datos-invalidos
El vecino ya fue atendido en otra delegación	10-alt-duplicidad
Rechazar sin escribir por qué	11-alt-observacion-obligatoria
Intentar validar tu propia evidencia	12-alt-validacion-propia
Dos personas guardando lo mismo a la vez	13-alt-conflicto-version
Tu rol no alcanza ese dato	14-alt-alcance-vecino y 15-alt-alcance-actividad
Una delegación sin nadie medido	16-alt-sin-medicion
Ver una atención de otra delegación	17-alt-alcance-reducido

Fíjate en un detalle de los dos «alcance»: cuando el sistema deniega algo, **escribe el motivo completo** en vez de un «sin permisos» a secas. No es cortesía: el alcance de estos datos es una decisión legal, y quien la recibe tiene derecho a saber por qué.

7. Los datos, en cristiano

El modelo tiene **22 tablas** y **52 claves foráneas**. No hace falta memorizarlas; sí conviene tener el mapa mental:

Configuración — quién mide qué `organizations` · `users` · `organization_members` (el rol vive aquí) · `unidades_territoriales` (las delegaciones) · `cargos` · `items_medicion` · `periodos` · `parametros`

Medición — cuánto se le pide a cada uno `metas_item` (meta + ponderador por funcionario, ítem y período) · `ausencias`

Registro — el trabajo del día `actividades` · `evidencias` · `validaciones` · `atenciones_sociales` · `personas_usuarias` (los vecinos)

Agenda colectiva — los compromisos `tareas` (el tubo) · `categorias_gestion` · `tarea_historial` · `catalogo_items`

Trazabilidad `auditoria` (quién hizo qué, cuándo, con qué valor anterior y nuevo) · `comentarios` · `ajustes`

El diagrama completo, con tipos y cardinalidades, está en [der.pdf](#). Y las mismas 22 como clases con sus métodos, en [clases.pdf](#).

Tres cosas del modelo que no son obvias

1. `PersonaUsuaría` es **única por organización, no por delegación**. Ese único detalle es lo que permite detectar al vecino que pide lo mismo en cinco lugares. Si fuera única por delegación, el sistema tendría el mismo problema que la planilla.
2. **Ningún valor de negocio está escrito en el código**. El tope del 150%, los umbrales del semáforo, la ventana de duplicidad y el tamaño máximo de una foto salen de la tabla `parametros`, con vigencia por período. Se cambian sin tocar una línea.

3. **La bitácora no se puede editar ni borrar.** Hay disparadores en la base de datos que lo impiden. Lo mismo con el código de una actividad: una vez puesto, no cambia.

8. Con qué está hecho

Capa	Tecnología
Backend	Node.js · TypeScript estricto · Express 5
Base de datos	PostgreSQL 16 · Prisma
Tiempo real	Socket.io, con salas por delegación y por organización
Frontend	React 18 · TypeScript · Vite 7
Estilos	CSS3 plano con tokens propios — sin Tailwind ni kits de UI
Interacción	dnd-kit (arrastre) · ECharts (gráficos)
Identidad	Libre Franklin + General Sans en pantalla · Arial en lo impreso

Costo cero: ninguna dependencia ni servicio de pago en todo el proyecto.

Por qué CSS plano y no un framework. La identidad gráfica de la Municipalidad de La Serena es normativa: el rojo institucional #DB0032 , el heráldico #8A0007 , la tipografía y el contraste no son preferencias nuestras. Un framework de terceros habría que pelearlo en vez de usarlo.

9. Qué está construido y qué no

De los **38 requerimientos funcionales** oficiales: **23 implementados y verificados** · **8 parciales** · **7 fuera de alcance por decisión**.

Funciona de punta a punta: registrar actividad → código único → subir evidencia → validarla → que sume al puntaje → verlo en el semáforo y en el tablero. Más la configuración que lo alimenta (cargo → ítems → metas) y la trazabilidad del vecino entre delegaciones.

No existe todavía: la API de ajustes, comentarios y catálogos; las pantallas de administración (períodos, cargos, catálogos); las pruebas en marco formal (Jest/RTL) y CI; y el despliegue.

Cuatro desvíos declarados

Estos son los puntos donde **el requerimiento pide una cosa y el código hace otra**. Están escritos en los artefactos de la entrega a propósito: un documento que describe el código como si fuera el requisito vuelve invisible el incumplimiento.

	Qué pide	Qué hay
D-a	Que el Funcionario cree compromisos en el tubo	Está restringido a jefatura. El funcionario los mueve, pero no los crea

	Qué pide	Qué hay
D-b	Cuatro estados con historial de transiciones y alertas de plazo	Hay tres estados y solo se marcan los vencidos. La tabla del historial existe, pero la aplicación nunca escribe en ella
D-c	Trazabilidad consultable	Se audita todo, pero falta la pantalla para leer la bitácora
D-d	Integridad referencial completa	<code>periodos.cerrado_por_id</code> debería ser clave foránea y no lo es

10. Cómo se comprueba que funciona

El proyecto no se apoya en «a mí me anda». Hay **332 comprobaciones automáticas** del software y **343** de coherencia entre los documentos de la entrega:

```
cd backend
npm run smoke           # 21 · integración
npm run verificar:calculo # 37 · el motor de cálculo
npm run verificar:api    # 191 · la API y el alcance de cada rol

cd frontend
npm run verificar:contraste # 83 · contraste WCAG en los dos temas
npm run verificar:mockups  # abre los 17 .html sin red
npm run verificar:entrega  # 343 · que los documentos digan lo mismo entre sí
```

Esa última es la que más cuida la nota: comprueba que la trazabilidad requerimiento → caso de uso → clase → tabla → pantalla cuadre **en los dos sentidos**, que ninguna clase o tabla esté inventada, y que las 52 claves foráneas del diagrama existan una por una en el script SQL.

11. Si quieres levantarlo en tu máquina

Necesitas **Docker** y **Node.js** (probado en la 24). Reserva **media hora**: el sembrado de datos solo tarda unos 20 minutos, porque genera 2.058 imágenes de evidencia una por una.

```

docker compose up -d          # PostgreSQL 16

cd backend
npm install
npx prisma migrate deploy
npx prisma db seed            # ~20 min sin imprimir nada. NO está colgado
npm run dev                   # API en :4000

cd ../frontend
npm install
npm run dev                   # aplicación en :5173

```

Entra a <<http://localhost:5173>> con cualquiera de estas seis, que cubren los seis roles. **Contraseña de todas:** `matriz123`.

Cuenta	Rol	Qué verás distinto
<code>admin@sgr.demo</code>	Administrador	Todo
<code>coordinador@sgr.demo</code>	Coordinador	Metas y panel de actividad
<code>verificador@sgr.demo</code>	Verificador	Solo la bandeja; sin tubo ni vecinos
<code>consulta@sgr.demo</code>	Consulta	Solo tableros
<code>delegado.centro@sgr.demo</code>	Delegado	Su delegación
<code>territorial.centro@sgr.demo</code>	Funcionario	Su ficha y su tubo

Son **23 cuentas en total** y todos los datos son **ficticios**, incluidas las fotos de evidencia, que las genera el propio sembrado. La lista completa está en [estado-proyecto.md §1](#).

Prueba siempre con los seis roles. Esa costumbre ya detectó tres errores reales que el camino feliz no mostraba.

12. Dónde está cada cosa

Si buscas...	Está en
El informe de la entrega	informe.pdf — el documento paraguas, con las cuatro tablas de trazabilidad
Los 38 requerimientos	requerimientos.pdf
Los casos de uso	casos-uso-general.pdf y casos-uso-detalle.pdf
Las clases y los servicios	clases.pdf
El modelo de datos	der.pdf
El script SQL	script-sql.pdf y sgr-mysql.sql
Las pantallas	mockup-pantallas.pdf · mockup-17-pantallas.zip · docs/mockups/

Si buscas...	Está en
Los 32 diagramas, con su fuente editable	docs/entrega/puml/
Por qué se decidió cada cosa	decisiones-tecnicas.md — 15 decisiones con su motivo
El estado técnico al detalle	estado-proyecto.md
Las normas del frontend	DESIGN.md

13. Lo que conviene tener claro para la defensa

Cinco preguntas probables y su respuesta corta:

«¿Por qué el DER está en MySQL si el sistema corre en PostgreSQL?» Porque la rúbrica pide MySQL. Se tradujo fielmente desde el esquema real, declarando las equivalencias de tipos, y **el script se ejecutó de verdad** en MySQL 8.0 y en MariaDB 10.4 y 11.4 —la de XAMPP— con cero errores. No se migró el sistema: cambiar la arquitectura de un producto que funciona para aprobar un criterio de formato sería el error caro.

«¿Por qué no hay herencia en el diagrama de clases?» Porque no la hay en el diseño. Los seis roles son valores de un enumerado que se **solapan**, no una jerarquía, y las entidades no comparten ancestro. La rúbrica pide herencia *con justificación*; afirmarla sin tenerla sería la incoherencia que el criterio transversal castiga.

«¿Cómo saben que los documentos dicen lo mismo entre sí?» Porque lo comprueba un script, no el ojo: 343 verificaciones que cruzan requerimientos, casos de uso, clases, tablas y pantallas en los dos sentidos.

«¿El mockup es un dibujo?» No. Es la aplicación real corriendo, congelada en archivos autocontenidos. Los 403 y el 409 que se ven en las pantallas los devolvió el servidor.

«¿Qué falta?» Está dicho en el informe: 7 requerimientos fuera de alcance con su motivo, 4 desvíos declarados y 14 consultas abiertas al docente. Ninguna ambigüedad se resolvió en silencio.

Equipo Origami SpA · Proyecto Integrador INACAP · <<https://github.com/Tibacache/matriz-sgr>>